

华中科技大学

软件课程设计报告（个人）

智能标注系统的设计与开发

院 系 电子信息与通信学院

专 业 通信工程

班 级 通信 2301 班

姓 名 毕嵩尧

学 号 U202314558

指导教师 王玉明

日 期 2026 年 1 月 10 日

2024 年 11 月制

目录

1. 项目概述	1
1.1 项目背景	1
1.2 相关技术	1
1.2.1 后端技术栈	1
1.2.2 前端技术栈	2
1.2.3 数据库技术	2
1.2.4 AI 服务技术	2
1.2.5 架构设计模式	2
1.3 设计任务与要求	3
2. 系统分析与设计	4
2.1 系统需求分析	4
2.1.1 功能需求分析	4
2.1.2 非功能需求分析	5
2.2 系统功能框架	6
2.2.1 文本管理模块	6
2.2.2 标注管理模块	6
2.2.3 算法管理模块	6
2.2.4 统计展示模块	6
2.2.5 字典管理模块	7
2.3 系统技术框架	7

2.3.1 后端技术框架.....	9
2.3.2 前端技术框架.....	9
2.3.3 AI 服务集成.....	10
2.4 系统原型设计	10
2.4.1 前端界面安排思路.....	10
2.4.2 后端交互逻辑.....	12
3. 系统开发与实现	12
3.1 前端开发环境	15
3.2 前端数据结构	15
3.2.1 文本数据模型 (TextDTO)	15
3.2.2 标注数据模型 (AnnotationDTO)	15
3.2.3 算法配置模型 (AlgorithmDTO)	16
3.2.4 统计数据结构.....	16
3.3 前端模块 A: 文本管理模块 (TEXTMANAGE)	16
3.3.1 模块层次关系.....	16
3.3.2 类继承关系.....	17
3.3.3 功能流程图.....	17
3.4 前端模块 B: 标注管理模块 (ANNOTATION)	18
3.4.1 模块层次结构.....	18
3.4.2 类继承关系.....	18
3.4.3 功能流程图.....	18
3.5 前端模块 C: 算法管理模块 (ALGORITHMMANAGE)	19

3.5.1 模块层次结构.....	19
3.5.2 功能流程图.....	20
3.6 前端模块 D: 统计展示模块 (STATISTICS)	20
3.6.1 模块层次结构.....	20
3.6.2 功能流程图.....	21
3.7 后端开发环境	21
3.8 后端数据结构	21
3.9 后端模块 A: 文本管理模块	22
3.9.1 模块层次关系.....	22
3.9.2 类继承关系.....	22
3.9.3 功能流程图.....	23
3.10 后端模块 B: 标注管理模块	23
3.10.1 模块层次结构.....	23
3.10.2 类继承关系.....	24
3.10.3 功能流程图.....	24
3.11 后端模块 C: 算法管理模块	25
3.11.1 模块层次结构.....	25
3.11.2 功能流程图.....	26
3.12 后端模块 D: 统计服务模块	26
3.12.1 模块层次结构.....	26
3.12.2 功能流程图.....	27
3.13 数据库开发环境	27

3.14 数据库数据结构	27
3.14.1 <i>original_text</i> 表（原始文本表）	27
3.14.2 <i>annotation</i> 表（标注信息表）	28
3.14.3 <i>annotation_algorithm</i> 表（算法配置表）	28
3.14.4 <i>annotation_dict</i> 表（字典表）	29
3.15 数据库模块 A：原始文本表设计	29
3.15.1 模块层次关系	29
3.15.2 查询优化	29
3.15.3 功能流程图	30
3.16 数据库模块 B：标注信息表设计	30
3.16.1 模块层次结构	30
3.16.2 查询优化	30
3.16.3 功能流程图	30
3.17 数据库模块 C：算法配置表设计	31
3.17.1 <i>JSON</i> 规则配置结构	31
3.17.2 功能流程图	32
3.18 标注算法设计环境	32
3.19 算法设计数据结构	33
3.20 算法设计模块 A：规则引擎算法 (RULEBASEDANNOTATIONPROVIDER)	33
3.20.1 模块层次关系	33
3.20.2 类继承关系	33

3.20.3 功能流程图.....	34
3.21 算法设计模块 B: DEEPSEEK API 算法 (DEEPSEEKAPIANNOTATIONPROVIDER)	35
3.21.1 模块层次结构.....	35
3.21.2 类继承关系.....	35
3.21.3 功能流程图.....	35
3.22 算法设计模块 C: 算法工厂 (ANNOTATIONPROVIDERFACTORY) ..	36
3.22.1 模块层次结构.....	36
3.22.2 功能流程图.....	36
3.23 算法设计模块 D: 规则引擎服务 (RULEENGINESERVICE) ..	36
3.23.1 模块层次结构.....	36
3.23.2 功能流程图.....	36
3.24 系统实现结果	37
后端开发 (Spring Boot)	37
4. 总结与建议.....	47
4.1 设计总结	
4.1.1 系统架构设计.....	
4.1.2 核心功能实现.....	
4.1.3 技术亮点.....	
4.1.4 系统优势.....	
4.1.5 学习收获.....	
4.2 相关建议	

参考文献	44
附录	45

1. 项目概述

本系统是基于 **SpringBoot + Vue2 + MySQL** 的 B/S 架构文本标注系统。支持文本上传、编辑与查询，提供人工标注与自动标注两种方式。自动标注支持规则引擎（正则表达式、关键词匹配）和 **AI 智能标注（DeepSeek API）**，覆盖分词、词性、命名实体等 **10 种** 标注类型。系统采用提供者模式，支持算法自定义管理，提供数据可视化统计，适用于 **NLP 模型训练、知识图谱构建** 等场景。

1.1 项目背景

在人工智能技术迅猛发展的核心，自然语言处理（**NLP**）已成为推动人机交互变革的核心引擎。从情感分析到知识图谱构建，**NLP 模型** 的性能往往建立在训练数据的质量与规模上。传统的标签标注 workflow 主要依赖人工逐条处理语言，这种方式不仅是定时消耗力，而且在面对海量数据时，人工标注的一致性和效率难以保证，成为避免模型迭代的“数据短缺”与此同时，随着大语言模型（**LLM**）的崛起，特别是 **DeepSeek** 等高性能模型的出现，利用 **AI 辅助标注** 已成为解决这一痛点的关键路径。

本项目旨在设计并开发一套人工标注与自动化标注于一体的智能文本标注系统。该系统不仅提供传统的基础文本管理功能，更创新性地融合了基于规则引擎的确定性标注与基于 **DeepSeek API** 的生成式 **AI 标注**，形成“规则+AI”的双模驱动架构。通过构建 **B/S 架构** 的协作平台，本系统致力于提升高质量 **NLP 数据集** 的生产，为简化的训练模型任务提供标准化、精细化的数据支撑

1.2 相关技术

本系统采用前后端分离的 **B/S 架构**，基于主流开源技术栈构建。后端采用 **Spring Boot** 框架提供 **RESTful API 服务**，前端采用 **Vue.js** 构建单页应用，数据库使用 **MySQL** 存储数据，并集成 **AI 服务** 实现智能标注。整体架构清晰、技术成熟，兼顾开发效率和系统性能，支持快速迭代和功能扩展。

1.2.1 后端技术栈

Spring Boot 3.2.0: 作为后端核心框架，提供自动配置、起步依赖和内置 **Tomcat**，简化开发与部署。内置依赖管理和健康检查，支持快速构建 **RESTful API**，满足系统业务逻辑处理需求。

MyBatis Plus 3.5.7: 在 MyBatis 基础上增强, 提供通用 CRUD、条件构造器、分页插件等, 减少重复代码。支持 Lambda 表达式查询, 提升开发效率, 简化数据库操作。

MySQL 8.0: 关系型数据库, 支持 JSON 类型存储标注元数据, utf8mb4 字符集支持中文, 事务保证数据一致性, 适合存储文本和标注数据。

1.2.2 前端技术栈

Vue 2.6.14: 渐进式 JavaScript 框架, 组件化开发, 数据驱动视图, 响应式数据绑定, 便于构建交互式界面。

Element UI: 基于 Vue 的组件库, 提供表格、表单、对话框等组件, 统一 UI 风格, 提升开发效率。

ECharts 5.4.3: 数据可视化库, 支持饼图、柱状图、折线图等, 用于展示标注类型分布和统计信息, 直观呈现数据。

1.2.3 数据库技术

MySQL 8.0: 采用关系型数据库设计, 核心表包括原始文本表、标注信息表、标注字典表和算法配置表。使用 JSON 类型存储复杂标注元数据, 支持事务保证数据一致性, 索引优化查询性能。

1.2.4 AI 服务技术

DeepSeek API: 集成 DeepSeek AI 模型, 提供基于语义理解的智能标注。支持多种标注类型, 通过 API 调用实现智能文本分析, 结合规则引擎形成双模式标注方案, 兼顾效率和准确性。

1.2.5 架构设计模式

提供者模式 (Provider Pattern): 用于实现多种自动标注方式。定义统一的标注提供者接口, 规则引擎提供者和 DeepSeek API 提供者分别实现, 便于扩展新的标注方式, 提高系统灵活性和可维护性。

前后端分离架构：前端负责用户界面和交互，后端提供 RESTful API，通过 HTTP 通信，降低耦合，支持独立开发和部署，提升开发效率和系统可扩展性。

1.3 设计任务与要求

第一，实现完整的文本管理功能，支持 txt、csv 等格式文件的上传、存储、编辑和查询，提供文本状态管理机制。

第二，提供双模式标注能力，支持人工标注和自动标注两种方式，自动标注包括基于规则引擎的快速标注和基于 AI 模型的智能标注，满足不同场景需求。

第三，支持多种标注类型，包括分词、词性、命名实体、关系、分类、情感、意图、语义、事件、错误等 10 种标注类型，每种类型使用不同颜色高亮显示

第四，实现算法管理功能，支持自定义规则算法配置，包括正则表达式和关键词匹配两种算法类型，提供算法的增删改查和启用/禁用功能。

第五，提供数据可视化展示，通过饼图、柱状图等图表直观展示标注数据分布和统计信息。系统要求具备良好的扩展性、易用性和稳定性，支持多用户并发使用，响应时间控制在 2 秒以内，文件上传支持最大 50MB。

2. 系统分析与设计

2.1 系统需求分析

随着自然语言处理技术的快速发展，文本标注作为 NLP 的基础环节，在模型训练、知识图谱构建、信息抽取等领域作用重要。传统人工标注存在效率低、成本高、一致性差等问题。

因此，需要开发一个集人工标注与自动标注于一体的文本标注系统。系统需支持多种格式文本文件的上传、存储、编辑和查询，提供人工标注和自动标注两种方式，支持分词、词性、命名实体、关系、分类、情感、意图、语义、事件、错误等 10 种标注类型。系统需支持自定义规则算法，包括正则表达式和关键词匹配，并集成 AI 模型提供基于语义理解的智能标注能力。系统需提供统计图表，直观展示标注数据分布，支持标注数据字典管理、标注结果可视化展示、批量标注处理和标注质量评估等扩展功能。

2.1.1 功能需求分析

（1）文本管理功能需求

系统需支持文本文件的上传、存储、编辑和查询。具体包括：支持 txt、csv 等格式文件上传；提供文本内容的在线编辑；实现文本列表的分页查询；管理文本状态（待标注、标注中、已完成），便于跟踪标注进度。

（2）标注功能需求

标注功能是核心，需支持人工标注和自动标注两种方式。人工标注：用户手动选择文本范围并添加标注信息。自动标注：提供规则引擎标注和 AI 智能标注两种模式。规则引擎标注基于正则表达式和关键词匹配，执行速度快，适合规则明确的场景；AI 智能标注基于 DeepSeek API，能够理解文本语义，适合复杂语义标注场景。系统需支持 10 种标注类型：分词、词性、命名实体、关系、分类、情感、意图、语义、事件、错误。不同标注类型需用不同颜色高亮显示，支持指定文本范围进行标注，并具备智能去重功能。

（3）算法管理功能需求

系统需提供算法配置管理功能，支持自定义规则算法的创建、编辑、删除和查询。算法类型包括正则表达式匹配和关键词匹配两种。系统需支持算法的启用和禁用，只有启用的算法才会在自动标注时生效。算法配置采用 JSON 格式存储，便于灵活配置和扩展。

（4）数据可视化功能需求

系统需提供数据统计和可视化展示功能，包括标注类型分布统计（饼图）、标注数量统计（柱状图）、数据概览卡片（文本总数、标注总数等），帮助用户直观了解标注数据分布情况。

2.1.2 非功能需求分析

（1）性能需求

系统响应时间要求：页面加载时间小于 2 秒，确保良好的用户体验。文件上传支持最大 50MB 的文件，满足实际使用需求。系统需支持多用户并发使用，保证在高并发场景下的稳定性。

（2）安全需求

数据存储安全：使用 MySQL 数据库，支持事务处理，确保数据一致性和完整性。API 安全：配置 CORS 跨域策略，防止未授权访问，保护系统安全。

（3）可用性需求

界面友好：基于 Element UI 组件库，提供统一的 UI 风格和良好的用户体验。操作简便：提供一键启动脚本，降低系统部署难度。错误提示：完善的错误提示和异常处理机制，帮助用户快速定位和解决问题。

2.2 系统功能框架

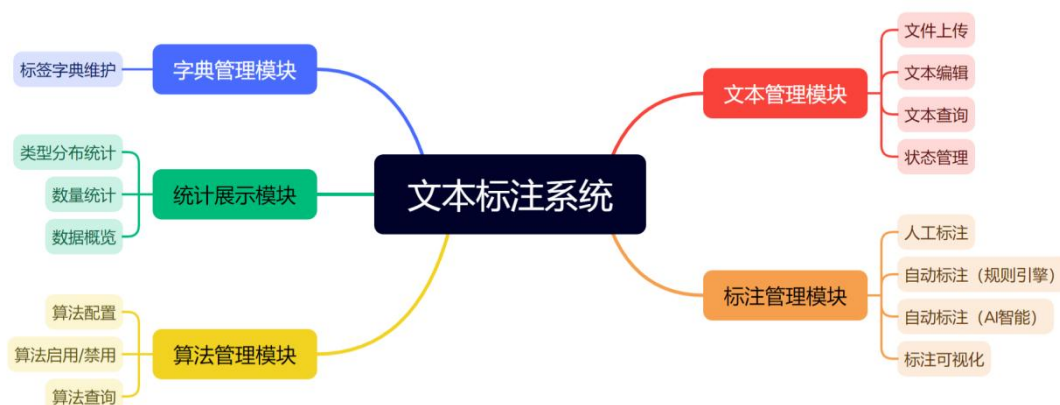


图 2.1 系统功能框架

2.2.1 文本管理模块

文本管理模块负责处理文本文件的上传、解析、存储和查询。该模块提供文件上传接口，支持多种格式文本文件的上传；提供文本内容的在线编辑功能；实现文本列表的分页查询和状态管理功能。

2.2.2 标注管理模块

标注管理模块是系统的核心模块，提供人工标注和自动标注两种方式。人工标注功能允许用户手动选择文本范围并添加标注信息；自动标注功能支持规则引擎标注和 AI 智能标注两种模式。该模块支持 10 种标注类型，提供标注的可视化展示、编辑和删除功能。

2.2.3 算法管理模块

算法管理模块负责管理规则算法配置，支持算法的增删改查和启用/禁用功能。该模块允许用户创建自定义规则算法，配置正则表达式或关键词匹配规则，并管理算法的启用状态。

2.2.4 统计展示模块

统计展示模块提供数据统计和可视化展示功能，包括标注类型分布统计、标注数量统计和数据概览信息，帮助用户直观了解系统的使用情况和标注数据分布。

2.2.5 字典管理模块

字典管理模块内部维护标注标签字典，确保标注的一致性和规范性。该模块为各类文本识别提供标准化的标签体系，包括词性、命名实体、情感等多种类型的字典数据。

2.3 系统技术框架

系统采用前后端分离的 B/S 架构，前端采用 Vue.js 构建单页应用，负责用户界面和交互；后端采用 Spring Boot 框架提供 RESTful API 服务，负责业务逻辑处理；数据库采用 MySQL 存储文本和标注数据。

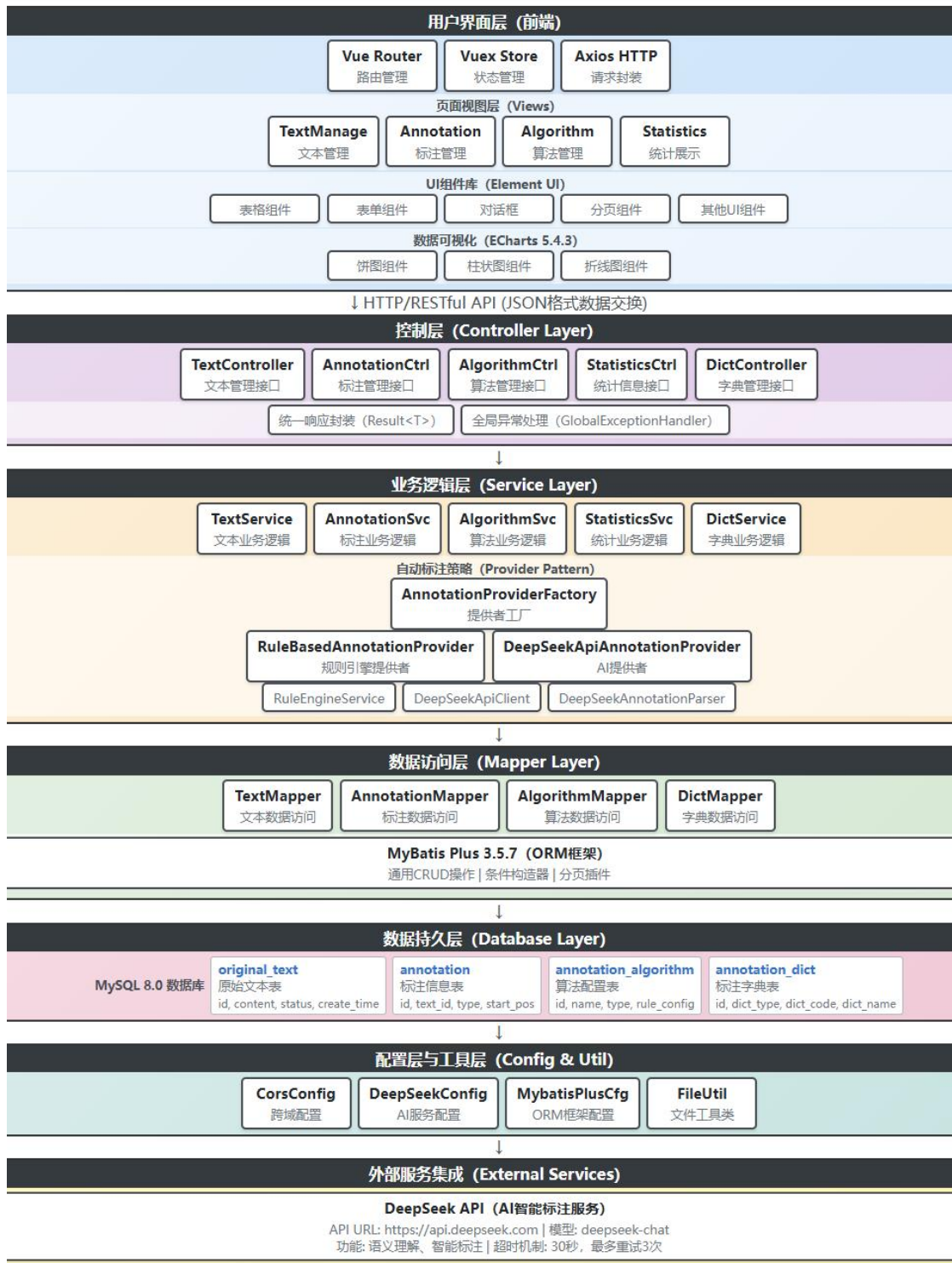


图 2.2 系统技术框架

前后端通过 HTTP 协议进行通信，实现了关注点分离，提高了系统的可维护性和可扩展性。

2.3.1 后端技术框架

（1）Spring Boot 3.2.0

Spring Boot 作为后端核心框架，提供自动配置、起步依赖和内置 Tomcat 服务器，简化了开发配置和部署流程。Spring Boot 的依赖管理机制和健康检查功能，为系统提供了良好的开发体验和运行保障。

（2）MyBatis Plus 3.5.7

MyBatis Plus 在 MyBatis 基础上进行了增强，提供了通用 CRUD、条件构造器、分页插件等功能，大大简化了数据库操作代码。支持 Lambda 表达式查询，提高了开发效率和代码可读性。

（3）MySQL 8.0

MySQL 8.0 作为关系型数据库，支持 JSON 数据类型，适合存储标注元数据；支持 utf8mb4 字符集，确保中文数据的正确存储；支持事务处理，保证数据一致性和完整性。

（4）提供者模式设计

系统采用提供者模式（Provider Pattern）实现多种自动标注方式。定义了统一的标注提供者接口，规则引擎提供者和 DeepSeek API 提供者分别实现该接口，便于扩展新的标注方式，提高了系统的灵活性和可维护性。

2.3.2 前端技术框架

（1）Vue 2.6.14

Vue.js 作为渐进式 JavaScript 框架，采用组件化开发模式，实现了数据驱动视图的响应式数据绑定，便于构建交互式用户界面。

（2）Element UI

Element UI 是基于 Vue 的组件库，提供了丰富的 UI 组件，包括表格、表单、对话框等，统一了 UI 风格，提高了开发效率。

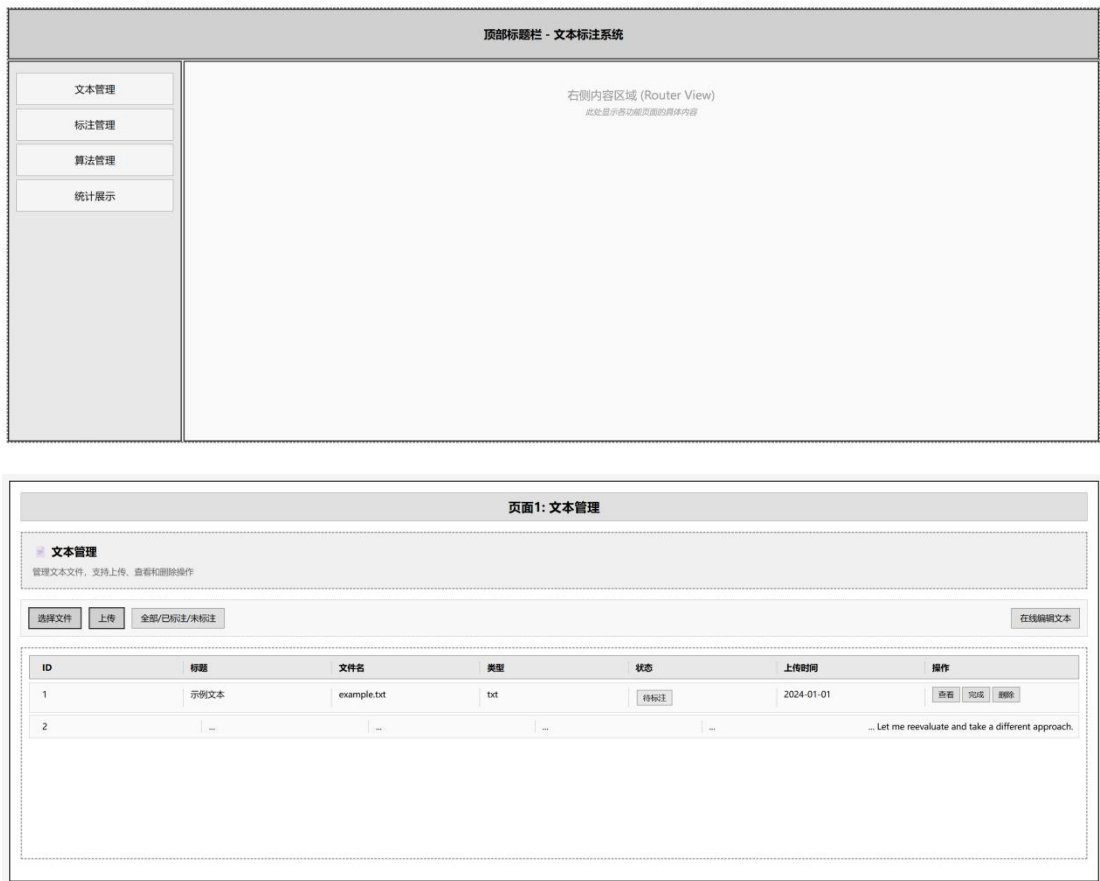
(3) ECharts 5.4.3

ECharts 作为数据可视化库，支持多种图表类型，包括饼图、柱状图、折线图，用于展示标注类型分布和统计信息，直观呈现数据。

2.3.3 AI 服务集成

系统集成 DeepSeek API，提供基于语义理解的智能标注能力。DeepSeek API 支持多种标注类型，通过 API 调用实现智能文本分析，结合规则引擎形成双模式标注方案，兼顾效率和准确性。

2.4 系统原型设计



2.4.1 前端界面安排思路

系统采用经典的后台管理布局，顶部固定标题栏、左侧可折叠导航菜单、右侧动态内容区。顶部标题栏高度 70px，展示系统名称和图标，使用深蓝色渐变背景与轻微动画。左侧导航宽度 240px（折叠后 64px），深色主题（#2c3e50），

包含四个功能模块：文本管理、标注管理、算法管理、统计展示，支持点击折叠展开。右侧内容区为自适应布局，基于 **Vue Router** 动态加载页面组件，页面切换采用淡入淡出动画。整体配色以深蓝色系为主，背景为浅灰渐变，采用卡片式设计 with 圆角，悬停时有阴影与轻微位移。

文本管理页面顶部为功能描述区域，操作栏包含文件上传按钮、在线编辑按钮和标注状态筛选（全部/已标注/未标注三级切换），下方以表格展示文件列表，包含 ID、标题、文件名、类型、状态标签和上传时间等字段，每行提供查看、完成、删除按钮，底部为分页组件。标注管理页面是核心功能页面，顶部工具栏左侧为文本选择下拉框，右侧为标注类型筛选下拉框（每种类型配有颜色标识方块），中间区域为手动标注按钮和自动标注选项（单选自研算法或 **DeepSeek API**）及执行按钮。文本显示区域采用等宽字体，支持根据标注列表自动高亮显示不同类型的标注内容（每种标注类型使用不同背景色，如分词为淡蓝色、词性为淡黄色、命名实体为淡绿色等），对于 **CSV** 文件支持分页加载机制（每次加载 **10** 行，滚动到底部可继续加载）。文本显示区域下方为标注列表表格，展示所有标注的 ID、类型、内容、标签、起始位置、结束位置等信息，每行提供编辑和删除操作。系统设计了多个对话框组件，包括手动标注对话框（用于选择标注类型和输入标签）、自动标注类型确认对话框和自动标注进度对话框（带进度条和状态提示文字，**DeepSeek API** 模式下会显示"正在整合本地文本"、"正在请求 **DeepSeek** 开放平台"等提示信息）。

算法管理页面顶部为筛选区域，包含算法类型筛选（规则/正则/关键词匹配）、标注类型筛选、状态筛选（启用/禁用）和排序方式选择，筛选条件可组合使用。筛选区域下方为算法列表表格，展示算法 ID、名称、类型、标注类型、状态（用标签颜色区分）和创建时间，每行提供编辑、启用/禁用切换和删除操作。点击添加或编辑按钮会弹出对话框，包含算法名称输入框、算法类型下拉选择、标注类型下拉选择、**JSON** 格式规则配置的文本域（支持正则表达式和关键词匹配两种配置方式）和状态选择。统计展示页面采用卡片式布局，顶部为页面标题和描述，上方区域包含两个 **ECharts** 图表，左侧为标注类型分布饼图（显示各类标注的比例关系），右侧为标注数量统计柱状图（横向展示各类型标注的绝对数量），

下方为三个统计卡片，分别展示总标注数、标注类型数和标注总数等关键指标，统计卡片采用大号数字和渐变文字效果。

2.4.2 后端交互逻辑

1. 总体通信架构

本项目采用了现代化的前后端分离架构，前端与后端之间通过标准的 **RESTful API (Representational State Transfer)** 进行通信。这种架构模式有效地解耦了视图层与业务逻辑层，使得前端专注于页面渲染与用户交互，后端专注于数据处理与业务实现。

所有的数据交互均基于 **HTTP/1.1** 协议，并强制使用 **JSON (JavaScript Object Notation)** 作为统一的数据交换格式。这种轻量级的数据格式不仅具备良好的可读性，还能被前后端语言高效地序列化与反序列化，确保了数据传输的高效性与兼容性。

2. 请求库封装与技术选型

前端采用了 **Axios** 作为核心 **HTTP** 客户端库。**Axios** 基于 **Promise** 设计，能够完美支持浏览器端和 **Node.js** 环境，具备强大的异步请求处理能力。

为了提高代码的可维护性和复用性，我们并未直接在组件中调用 **Axios**，而是对其进行了二次封装：

实例化管理：创建了全局唯一的 **Axios** 实例，统一管理请求配置。

模块化设计：将 **API** 请求按业务模块（如用户管理、标注任务、系统设置）进行分类封装，便于后续的接口维护与版本迭代。

3. 请求配置与性能策略

针对本项目的特定业务场景，我们在 **Axios** 实例中进行了以下关键配置：

基础路径 (Base URL)：请求的基础路径统一配置为 **http://localhost:8080/api**。在开发环境中，这指向本地启动的后端服务；

在生产部署时，可通过环境变量动态替换为线上网关地址，实现了环境配置的灵活切换。

超时策略 (Timeout): 考虑到系统中包含“自动标注”、“AI 模型推理”等计算密集型任务，这些操作通常耗时较长。为了防止前端在等待复杂逻辑处理时过早抛出连接超时错误，我们将全局请求超时时间设定为 50,000 毫秒（50 秒）。这一配置在保证用户体验的同时，给予了后端充足的时间来完成图像处理与算法运算。

4. 统一响应数据结构 (Data Contract)

为了降低前后端联调的沟通成本，后端对所有接口的返回数据进行了严格的标准化处理。无论请求成功与否，后端均返回一个结构固定的 **Result** 对象。该对象包含以下三个核心字段：

字段名	类型	描述
code	Integer	业务状态码。用于标识请求的处理结果（如：200 表示成功，401 表示未授权，500 表示系统异常）。此状态码与 HTTP 状态码逻辑分离，更专注于业务层面的状态表达。
message	String	提示信息。用于向用户展示操作结果的反馈文本（如“操作成功”、“参数校验失败”），便于前端直接展示或用于日志记录。
data	Object/T	数据载荷。当请求成功时，该字段承载具体的业务数据（如用户列表、标注结果）；当请求失败时，该字段通常为 空 。

5. 拦截器机制与异常处理

为了减少冗余代码并实现统一的逻辑控制，前端利用 **Axios** 的**拦截器 (Interceptors) **机制构建了完善的请求与响应处理流：

5.1 请求拦截 (Request Interceptor)

在请求发送前，拦截器会自动执行预处理逻辑（如添加 **Token** 身份凭证、序列化请求参数等），确保发出的请求符合后端安全规范。

5.2 响应拦截 (Response Interceptor)

在收到后端响应后，拦截器会根据 **Result** 对象中的 **code** 字段进行分流处理：

业务成功：当 **code** 表示成功时，拦截器会自动剥离外层的 **Result** 结构，直接将 **data** 字段的数据返回给具体的业务组件。这使得组件内部无需重复编写解包逻辑，能够直接使用核心数据。

业务失败与系统异常：当检测到非成功状态码或网络层面的异常（如 **404**、**500**、**Network Error**）时，拦截器会统一捕获错误。

它会调用全局 **UI** 组件（如 **Message** 或 **Notification**）向用户展示友好的错误提示信息（即 **message** 字段内容）。

对于特定错误（如 **Token** 过期），拦截器会自动触发路由跳转至登录页，实现了自动化的异常熔断与恢复机制。

3. 系统开发与实现

四位同学分别负责前端开发、后端开发、数据库设计、标注算法设计。

3.1 前端开发环境

开发工具：IDE：Visual Studio Code 1.80+、Node.js：v16.0+

包管理器：npm 8.0+

前端技术栈：Vue.js 2.6.14（MVVM）、Vue Router 3.6.5（路由）、Vuex 3.6.2（状态管理）、Element UI 2.15.14（UI）、Axios 1.6.0（HTTP）、ECharts 5.4.3（图表）

构建工具：Vue CLI 5.0.0、Babel 7.12.16（ES6+转译）、ESLint 7.32.0（代码检查）

浏览器支持：Chrome 90+、Firefox 88+、Edge 90+

3.2 前端数据结构

3.2.1 文本数据模型（TextDT0）

```
{
  id: Number,           // 文本 ID
  title: String,         // 文本标题
  content: String,       // 文本内容
  fileName: String,      // 文件名
  fileType: String,      // 文件类型
  status: String,        // 状态：待标注/标注中/已完成
  uploadTime: String    // 上传时间
}
```

3.2.2 标注数据模型（AnnotationDT0）

```
{
  id: Number,           // 标注 ID
  textId: Number,       // 关联文本 ID
  annotationType: String, // 标注类型
  startPos: Number,     // 起始位置
  endPos: Number,       // 结束位置
}
```

```

content: String,           // 标注内容
label: String,             // 标注标签
metadata: Object           // 扩展信息 (JSON)
}

```

3.2.3 算法配置模型 (AlgorithmDTO)

```

{
  id: Number,
  algorithmName: String,
  algorithmType: String, // 规则/正则/关键词匹配
  annotationType: String,
  ruleConfig: Object,    // 规则配置 (JSON)
  status: String         // 启用/禁用
}

```

3.2.4 统计数据结构

```

{
  textStatistics: { // 文本统计
    total: Number,
    pending: Number,
    annotating: Number,
    completed: Number
  },
  annotationStatistics: { // 标注统计
    byType: Object, // 按类型统计
    byLabel: Object // 按标签统计
  }
}

```

3.3 前端模块 A: 文本管理模块 (TextManage)

3.3.1 模块层次关系

TextManage.vue (页面组件)

- ├── API 层: text.js (API 接口封装)
 - | ├── getTextList() - 获取文本列表
 - | ├── uploadText() - 上传文本
 - | ├── deleteText() - 删除文本
 - | └── updateTextStatus() - 更新文本状态
- ├── 组件层: Element UI 组件
 - | ├── el-upload - 文件上传组件
 - | ├── el-table - 文本列表表格
 - | ├── el-pagination - 分页组件
 - | └── el-dialog - 编辑对话框

└── 工具层: request.js (HTTP 请求封装)

3.3.2 类继承关系

Vue (基类)

└── TextManage (页面组件)

└── 数据属性: textList, currentPage, total 等

└── 计算属性: filteredTextList

└── 方法: handleUpload, handleDelete, handleView 等

3.3.3 功能流程图

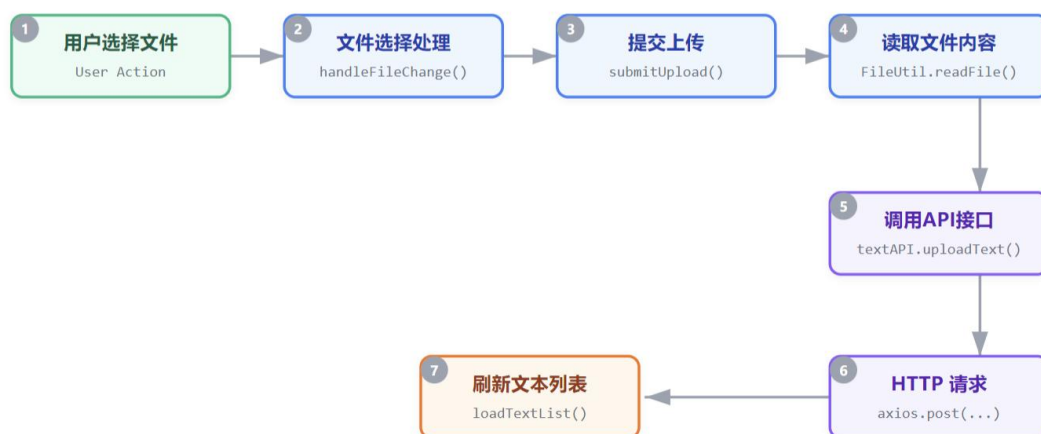


图 3.1 文件上传流程

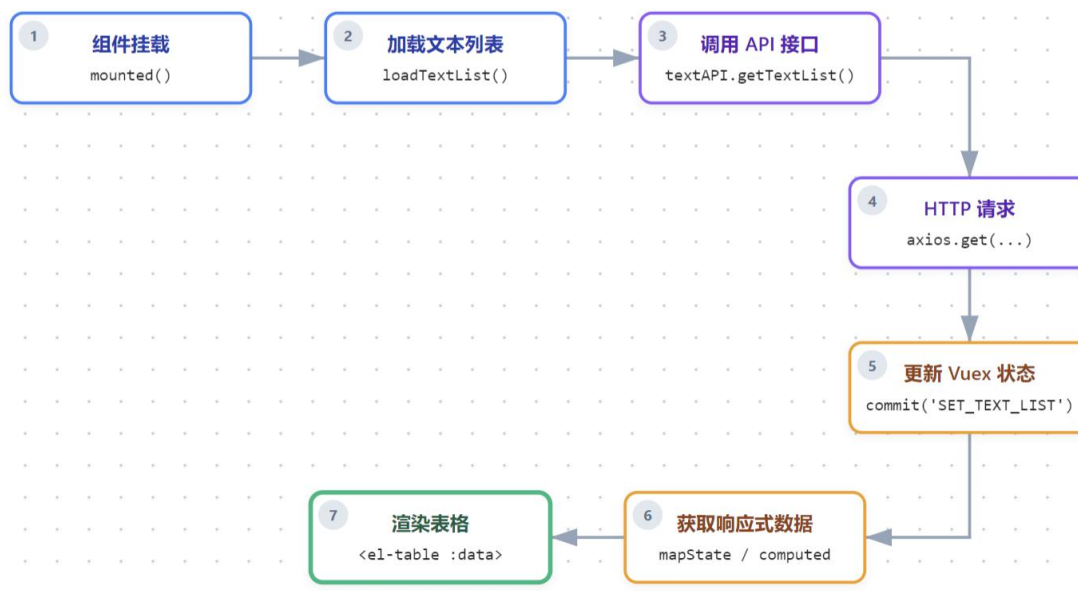
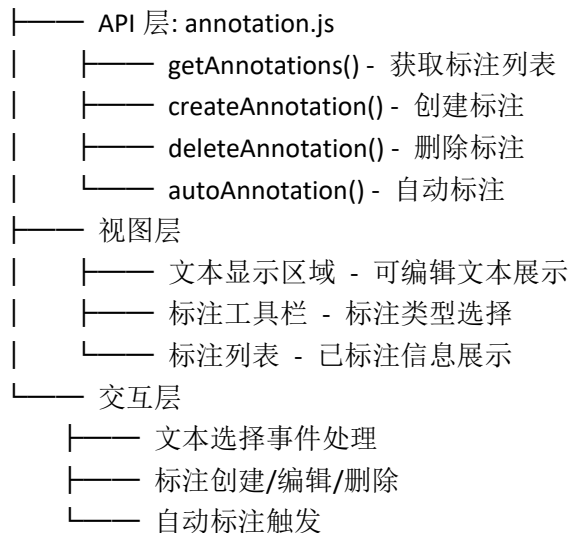


图 3.2 文本列表查询流程

3.4 前端模块 B：标注管理模块（Annotation）

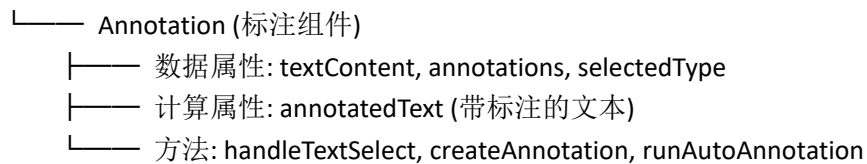
3.4.1 模块层次结构

Annotation.vue (标注页面)



3.4.2 类继承关系

Vue (基类)



3.4.3 功能流程图

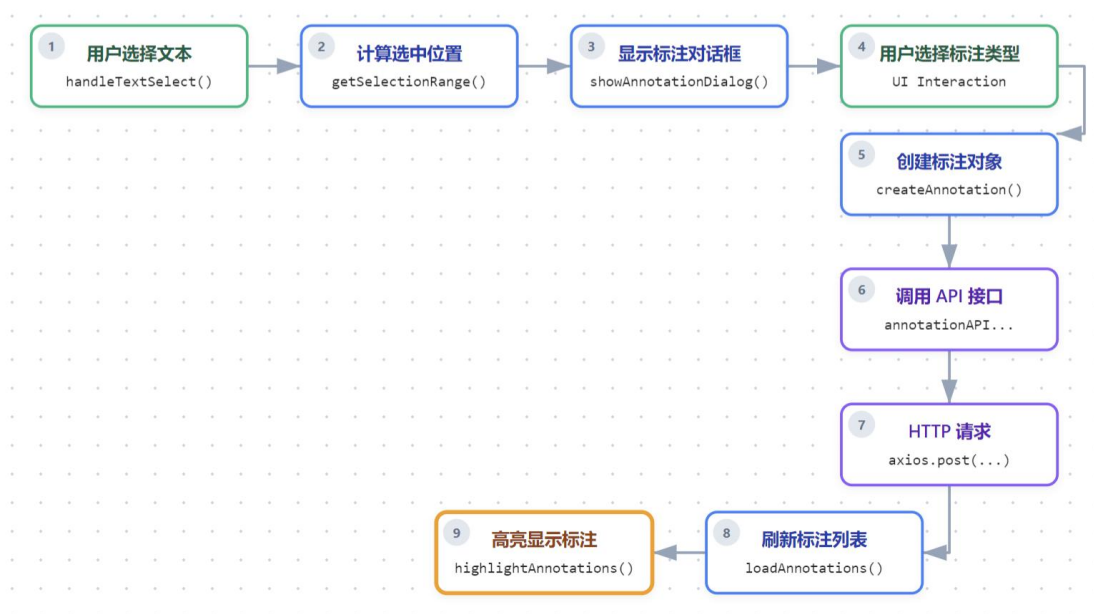


图 3.3 手动标注流程

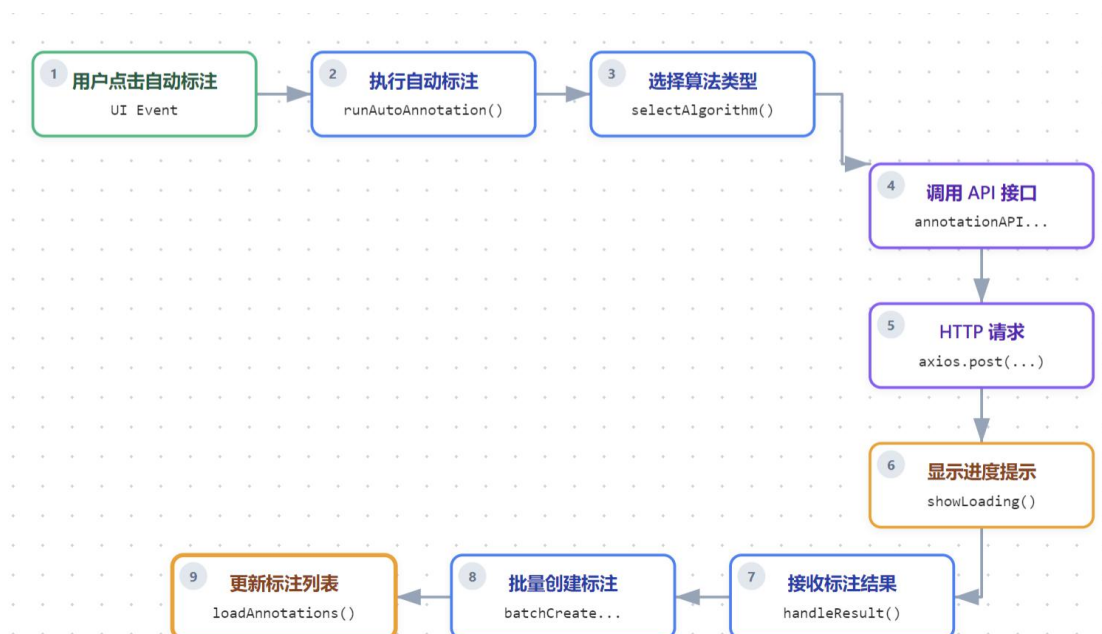


图 3.4 自动标注流程

3.5 前端模块 C：算法管理模块 (AlgorithmManage)

3.5.1 模块层次结构

```

AlgorithmManage.vue
├── API 层: algorithm.js
│   ├── getAlgorithmList() - 获取算法列表
│   ├── createAlgorithm() - 创建算法
│   ├── updateAlgorithm() - 更新算法
│   └── deleteAlgorithm() - 删除算法
├── 表单组件
│   ├── 算法基本信息表单
│   └── 规则配置 JSON 编辑器
└── 列表组件
    └── 算法列表表格 (支持启用/禁用)
    
```

3.5.2 功能流程图

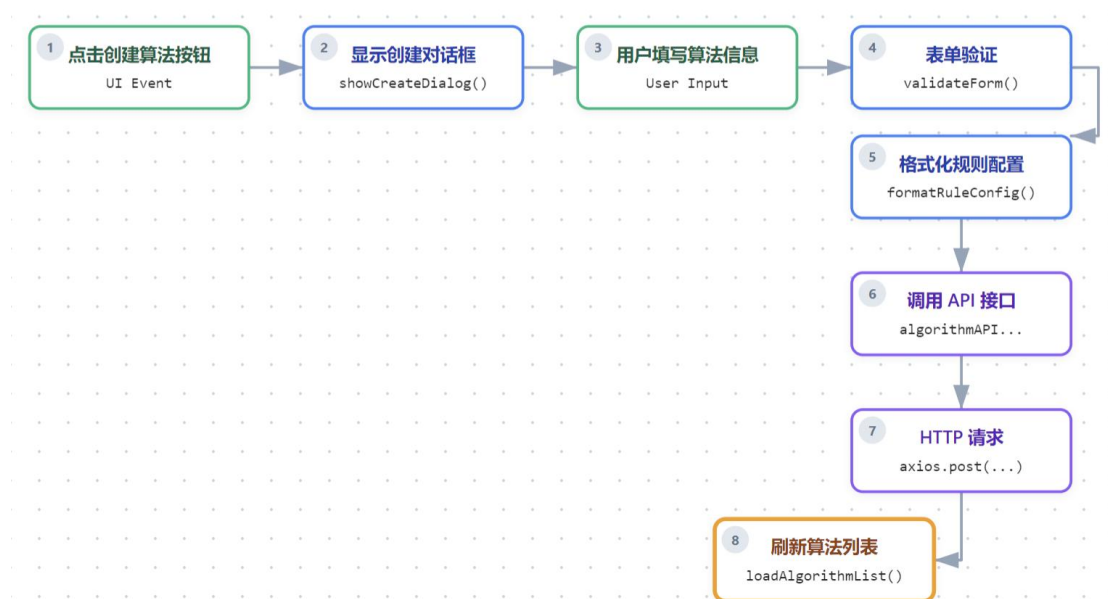


图 3.5 算法管理流程

3.6 前端模块 D：统计展示模块（Statistics）

3.6.1 模块层次结构

Statistics.vue

- ├── API 层: statistics.js
 - ├── getStatistics() - 获取统计数据
 - └── getAnnotationDistribution() - 获取标注分布
- ├── 图表组件: ECharts
 - ├── 饼图 - 标注类型分布
 - ├── 柱状图 - 标注数量统计
 - └── 折线图 - 时间趋势
- └── 数据展示组件
 - └── 统计卡片组件

3.6.2 功能流程图

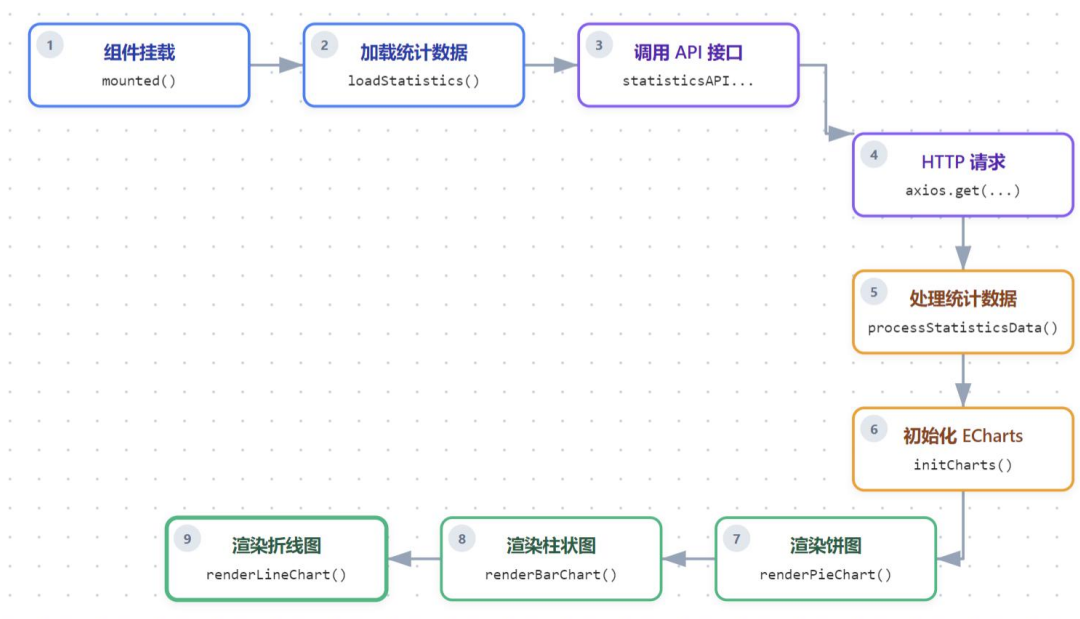


图 3.6 统计展示流程

3.7 后端开发环境

版本：Java 17

构建工具：Apache Maven 3.9.11

框架版本：Spring Boot 3.2.0

ORM 框架：MyBatis Plus 3.5.8

数据库驱动：MySQL Connector 8.0.33

工具库：Lombok 1.18.22、FastJSON 2.0.43

开发工具：IntelliJ IDEA、Postman

3.8 后端数据结构

```

// OriginalText 实体
- id: Long (主键)
- title: String
    
```

```
- content: String (LONGTEXT)
- fileName: String
- fileType: String
- status: String
- uploadTime: LocalDateTime
```

// Annotation 实体

```
- id: Long
- textId: Long (外键)
- annotationType: String
- startPos: Integer
- endPos: Integer
- content: String
- label: String
- metadata: JSON
```

// AnnotationAlgorithm 实体

```
- id: Long
- algorithmName: String
- algorithmType: String
- annotationType: String
- ruleConfig: JSON
- status: String
```

3.9 后端模块 A：文本管理模块

3.9.1 模块层次关系

```
TextController (控制层)
    ↓
TextService (业务逻辑层)
    ↓
TextMapper (数据访问层)
    ↓
OriginalText (实体类)
```

3.9.2 类继承关系

```
BaseMapper<OriginalText> (MyBatis Plus)
    ↑
TextMapper (接口)
    ↑
TextService (实现类)
    ↑
```

TextController (@RestController)

3.9.3 功能流程图

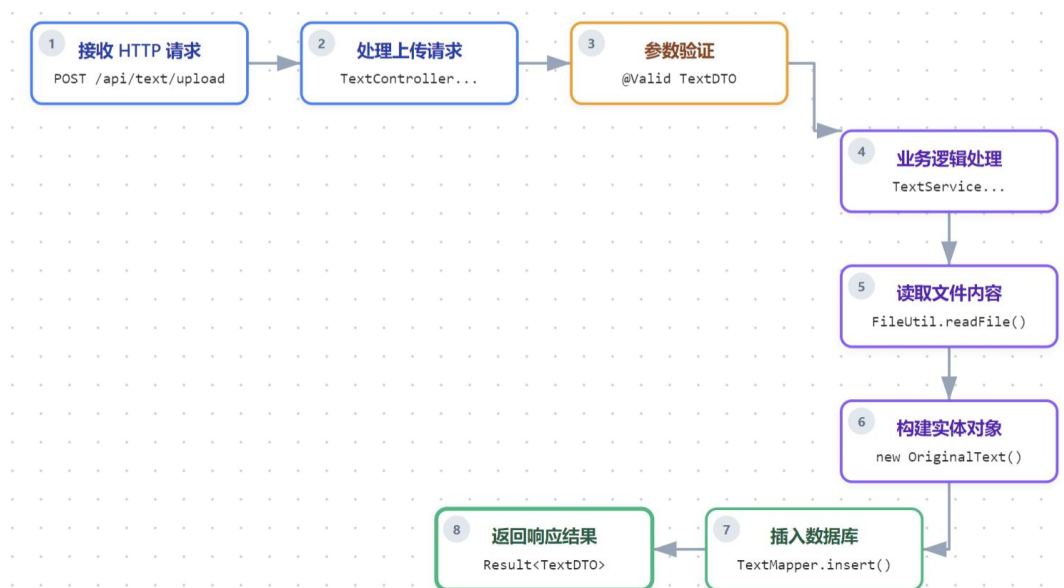


图 3.1 文本上传流程

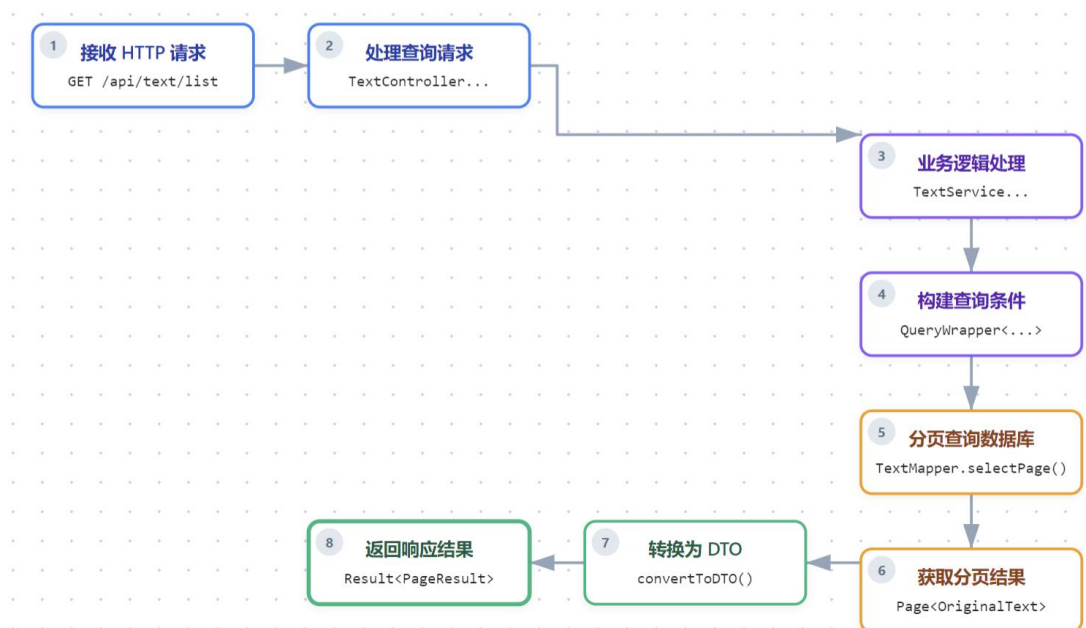
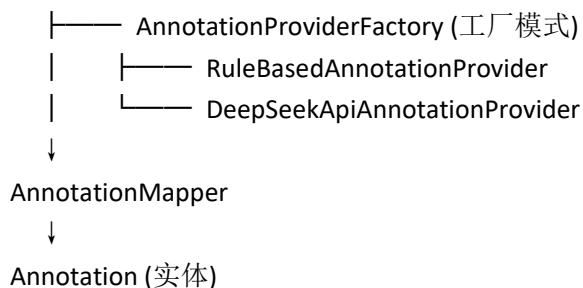


图 3.2 文本列表查询流程

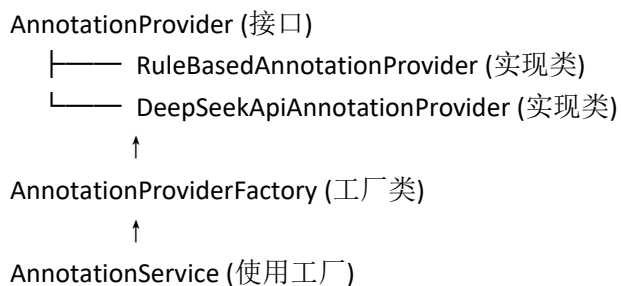
3.10 后端模块 B：标注管理模块

3.10.1 模块层次结构

AnnotationController
↓
AnnotationService



3.10.2 类继承关系



3.10.3 功能流程图

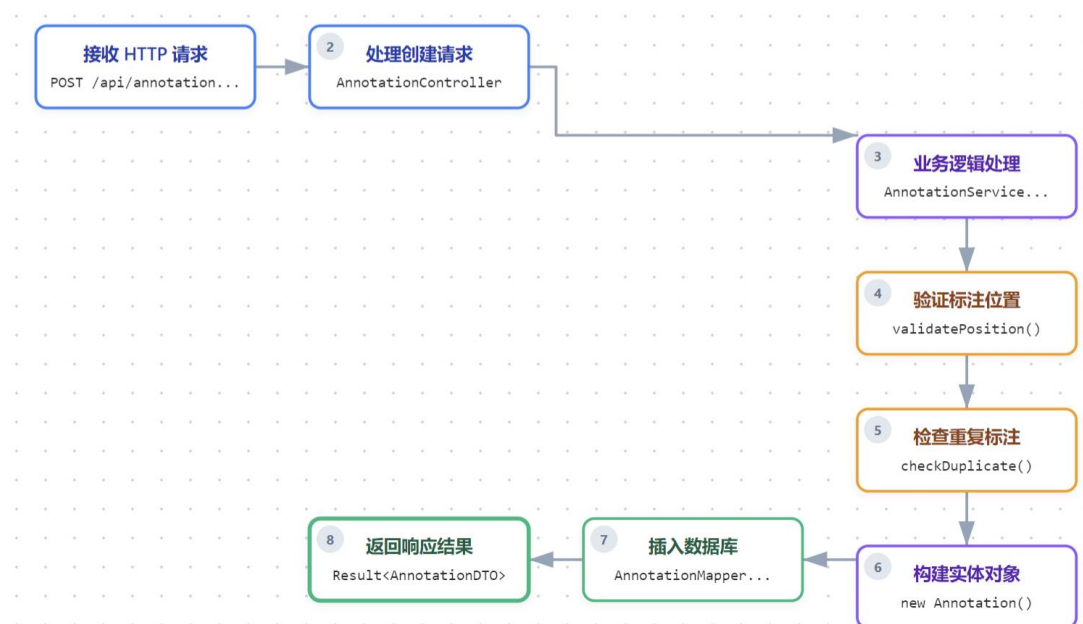


图 3.3 手动标注流程

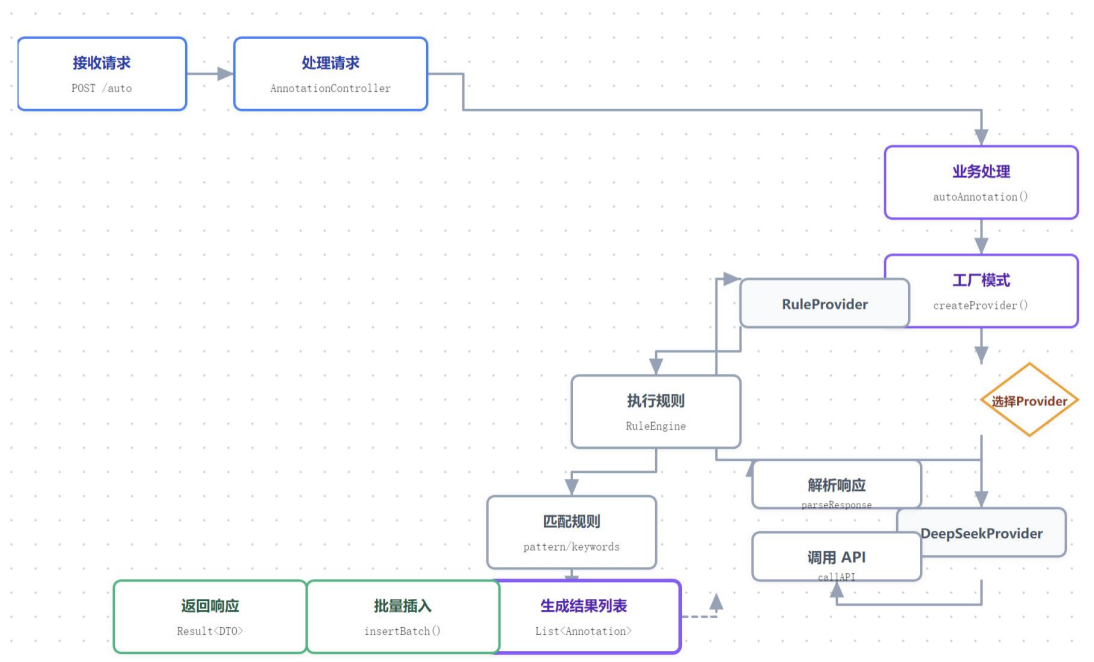
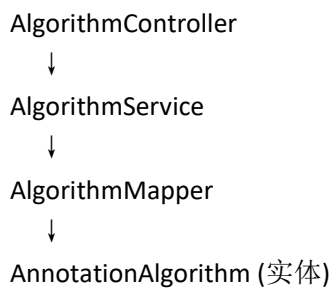


图 3.4 自动标注流程

3.11 后端模块 C：算法管理模块

3.11.1 模块层次结构



3.11.2 功能流程图

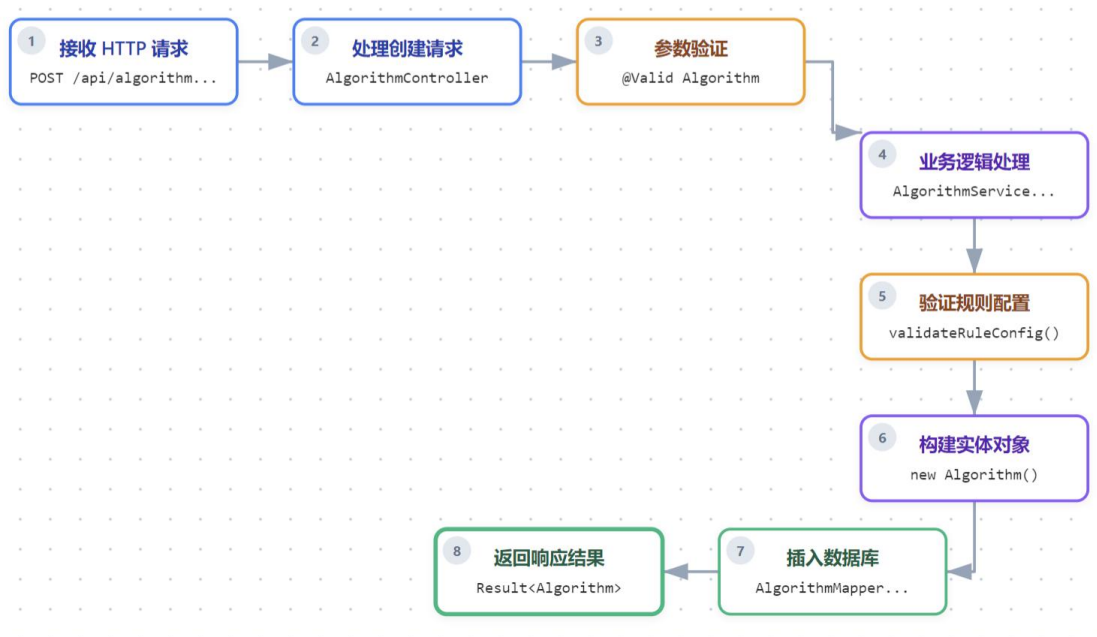
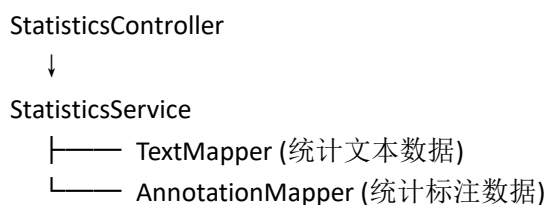


图 3.5 算法创建流程

3.12 后端模块 D：统计服务模块

3.12.1 模块层次结构



3.12.2 功能流程图

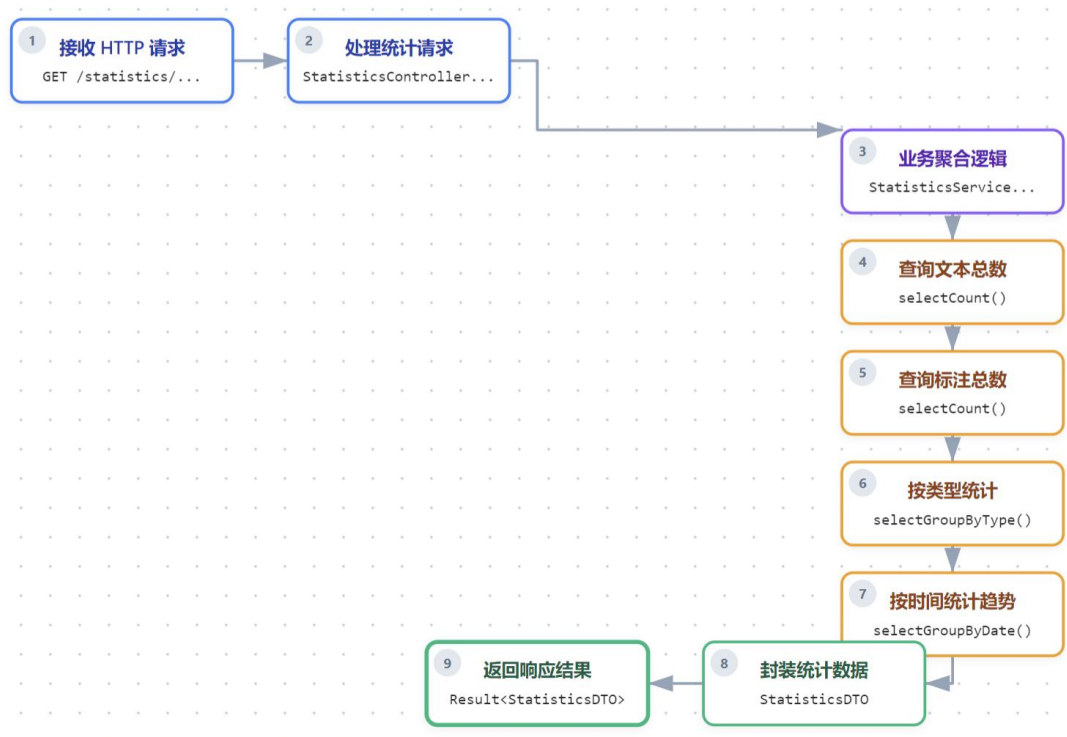


图 3.6 统计服务流程

3.13 数据库开发环境

数据库：MySQL 8.0

字符集：utf8mb4

设计工具：MySQL Workbench、Navicat

版本控制：SQL 脚本管理

3.14 数据库数据结构

3.14.1 original_text 表（原始文本表）

字段设计：

- id: BIGINT PRIMARY KEY AUTO_INCREMENT
- title: VARCHAR(255) NOT NULL
- content: LONGTEXT NOT NULL
- file_name: VARCHAR(255)

```
- file_type: VARCHAR(10)
- upload_time: DATETIME DEFAULT CURRENT_TIMESTAMP
- status: VARCHAR(20) DEFAULT '待标注'
- create_time: DATETIME DEFAULT CURRENT_TIMESTAMP
- update_time: DATETIME ON UPDATE CURRENT_TIMESTAMP
```

索引设计:

```
- PRIMARY KEY (id)
- INDEX idx_status (status)
- INDEX idx_upload_time (upload_time)
```

3.14.2 annotation 表 (标注信息表)

字段设计:

```
- id: BIGINT PRIMARY KEY AUTO_INCREMENT
- text_id: BIGINT NOT NULL (外键)
- annotation_type: VARCHAR(50) NOT NULL
- start_pos: INT NOT NULL
- end_pos: INT NOT NULL
- content: VARCHAR(500) NOT NULL
- label: VARCHAR(100) NOT NULL
- metadata: JSON
- annotator_id: BIGINT
- create_time: DATETIME DEFAULT CURRENT_TIMESTAMP
- update_time: DATETIME ON UPDATE CURRENT_TIMESTAMP
```

索引设计:

```
- PRIMARY KEY (id)
- FOREIGN KEY (text_id) REFERENCES original_text(id) ON DELETE CASCADE
- INDEX idx_text_id (text_id)
- INDEX idx_annotation_type (annotation_type)
- INDEX idx_label (label)
```

3.14.3 annotation_algorithm 表 (算法配置表)

字段设计:

```
- id: BIGINT PRIMARY KEY AUTO_INCREMENT
- algorithm_name: VARCHAR(255) NOT NULL
- algorithm_type: VARCHAR(50) NOT NULL
- annotation_type: VARCHAR(50) NOT NULL
- rule_config: JSON NOT NULL
- status: VARCHAR(20) DEFAULT '启用'
- create_time: DATETIME DEFAULT CURRENT_TIMESTAMP
- update_time: DATETIME ON UPDATE CURRENT_TIMESTAMP
```

索引设计:

- PRIMARY KEY (id)
- INDEX idx_annotation_type (annotation_type)
- INDEX idx_status (status)

3.14.4 annotation_dict 表 (字典表)

字段设计:

- id: BIGINT PRIMARY KEY AUTO_INCREMENT
- dict_type: VARCHAR(50) NOT NULL
- dict_code: VARCHAR(50) NOT NULL
- dict_name: VARCHAR(100) NOT NULL
- dict_value: VARCHAR(255)
- sort_order: INT DEFAULT 0
- create_time: DATETIME DEFAULT CURRENT_TIMESTAMP
- update_time: DATETIME ON UPDATE CURRENT_TIMESTAMP

索引设计:

- PRIMARY KEY (id)
- UNIQUE KEY uk_dict_type_code (dict_type, dict_code)
- INDEX idx_dict_type (dict_type)

3.15 数据库模块 A: 原始文本表设计

3.15.1 模块层次关系

original_text (1) ——< (N) annotation
主表 从表

3.15.2 查询优化

- 1.状态字段建立索引, 支持快速筛选
- 2.上传时间索引, 支持时间范围查询
- 3.使用分页查询减少数据量

3.15.3 功能流程图

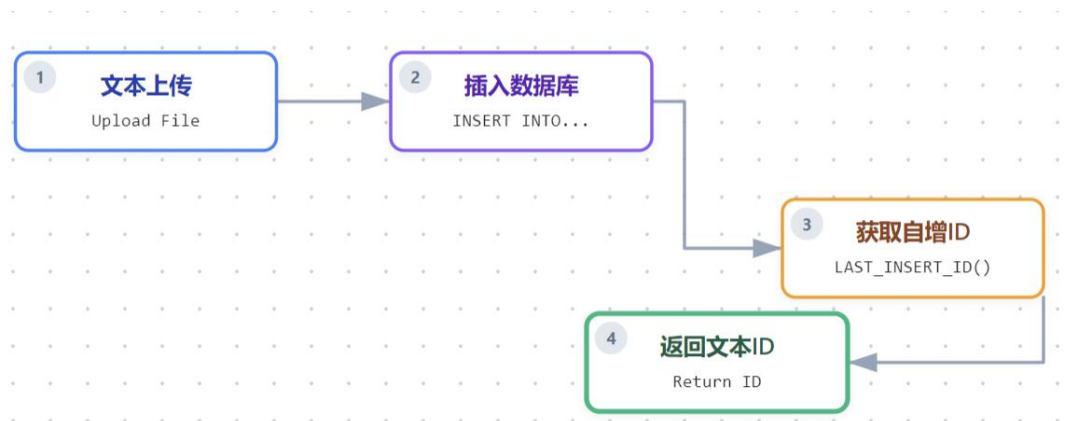


图 3.1 文件上传流程

3.16 数据库模块 B：标注信息表设计

3.16.1 模块层次结构

original_text (1) ——< (N) annotation
主键: id 外键: text_id

3.16.2 查询优化

1.text_id 索引：快速查询某文本的所有标注

2.annotation_type 索引：按类型筛选标注

3.复合查询优化：使用覆盖索引减少回表

3.16.3 功能流程图

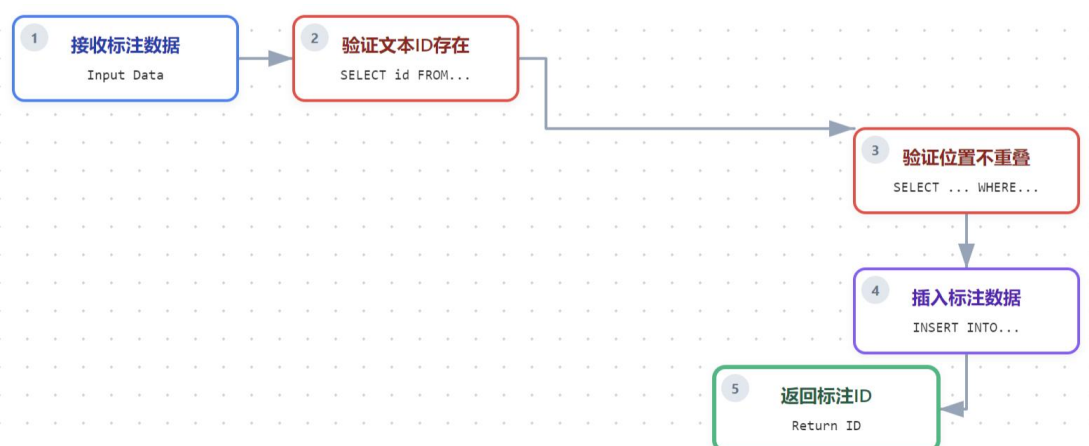


图 3.3 标注创建流程

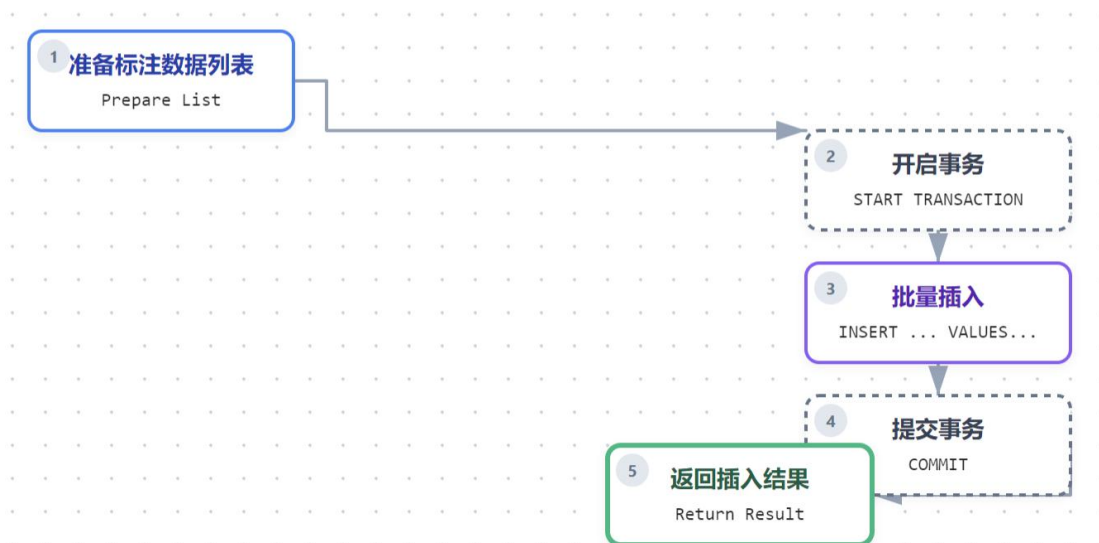


图 3.4 批量标注创建流程

3.17 数据库模块 C：算法配置表设计

3.17.1 JSON 规则配置结构

```

// 正则表达式类型
{
  "pattern": "正则表达式",
  "label": "标签名称"
}

// 关键词匹配类型
{
  "keywords": ["关键词 1", "关键词 2"],
  "label": "标签名称"
}
    
```

3.17.2 功能流程图



图 3.5 算法配置查询流程



图 3.6 算法配置更新流程

3.18 标注算法设计环境

编程语言：Java 17

正则引擎：Java Pattern/Matcher

JSON 处理：FastJSON 2.0.43

HTTP 客户端：Spring RestTemplate

API 服务：DeepSeek API (<https://api.deepseek.com>)

3.19 算法设计数据结构

```
// 规则配置JSON 结构
{
    "pattern": String,           // 正则表达式（正则类型）
    "keywords": String[],       // 关键词列表（关键词类型）
    "label": String             // 标注标签
}

// 标注结果结构
{
    textId: Long,
    annotationType: String,
    startPos: Integer,
    endPos: Integer,
    content: String,
    label: String
}
```

3.20 算法设计模块 A：规则引擎算法 (RuleBasedAnnotationProvider)

3.20.1 模块层次关系

```
RuleBasedAnnotationProvider (提供者实现)
    ↓
RuleEngineService (规则引擎服务)
    ↓
AlgorithmService (算法配置查询)
    ↓
AnnotationAlgorithm (算法实体)
```

3.20.2 类继承关系

```
AnnotationProvider (接口)
    ↑
RuleBasedAnnotationProvider (实现类)
    |—— 依赖: RuleEngineService
    |—— 依赖: AlgorithmService
```


3.20.3 功能流程图

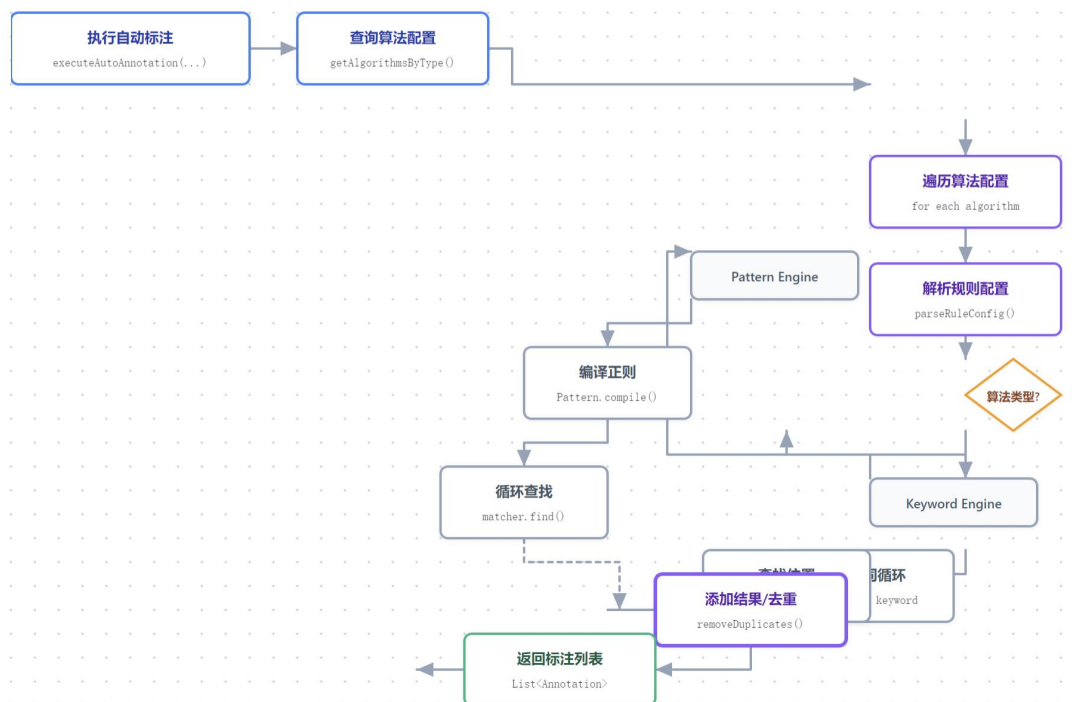


图 3.1 规则匹配流程

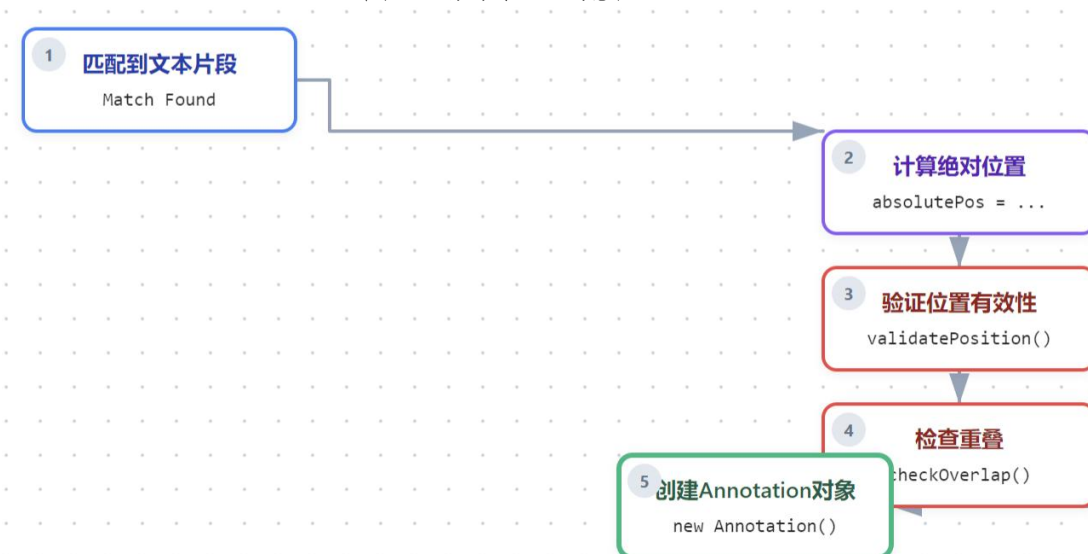
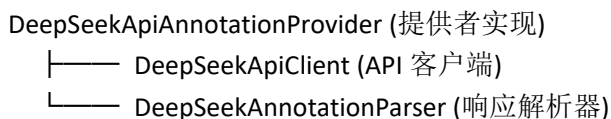


图 3.2 位置计算流程

3.21 算法设计模块 B：DeepSeek API 算法 (DeepSeekApiAnnotationProvider)

3.21.1 模块层次结构



3.21.2 类继承关系



3.21.3 功能流程图

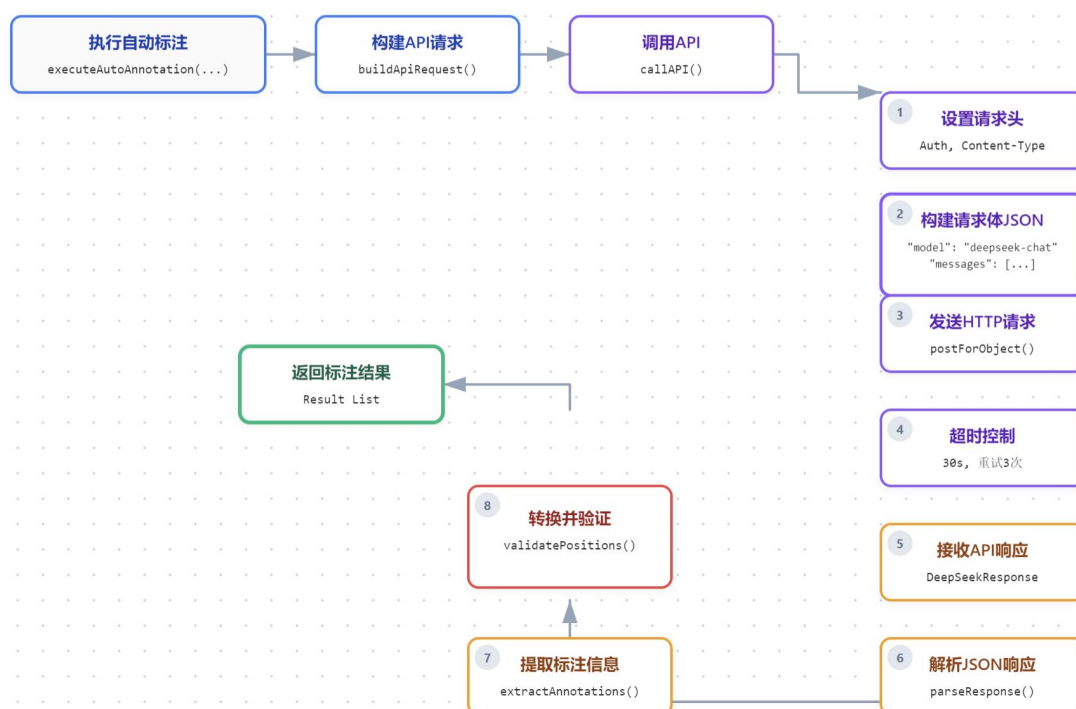


图 3.3 AI 标注流程

3.22 算法设计模块 C: 算法工厂 (AnnotationProviderFactory)

3.22.1 模块层次结构

AnnotationProviderFactory (工厂类)

- ├── RuleBasedAnnotationProvider (规则提供者)
- └── DeepSeekApiAnnotationProvider (AI 提供者)

3.22.2 功能流程图

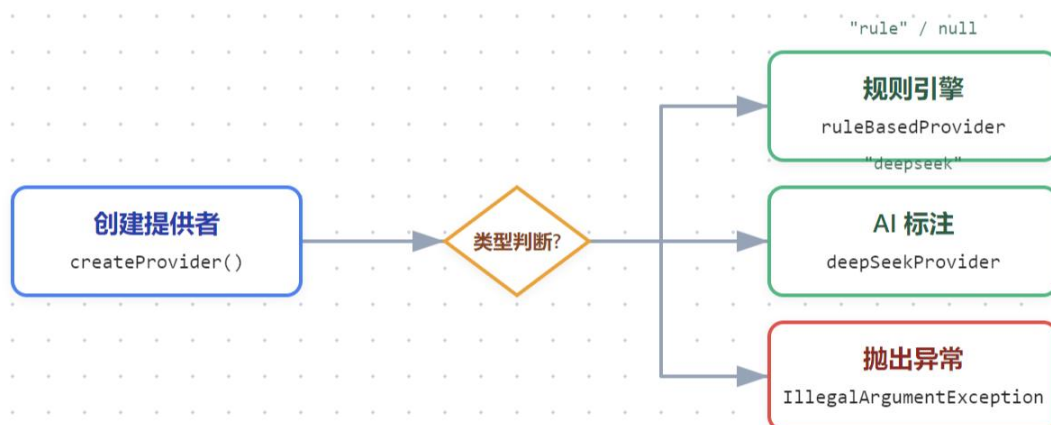


图 3.5 提供者创建流程

3.23 算法设计模块 D: 规则引擎服务 (RuleEngineService)

3.23.1 模块层次结构

RuleEngineService (规则引擎核心服务)

- ├── matchByPattern() - 正则匹配
- └── matchByKeywords() - 关键词匹配

3.23.2 功能流程图

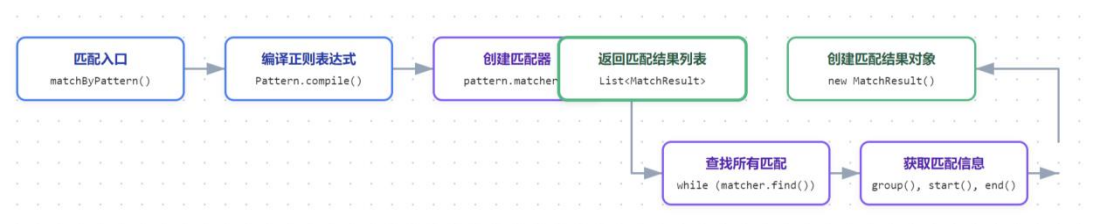


图 3.6 正则表达式匹配流程



图 3.7 关键词匹配流程

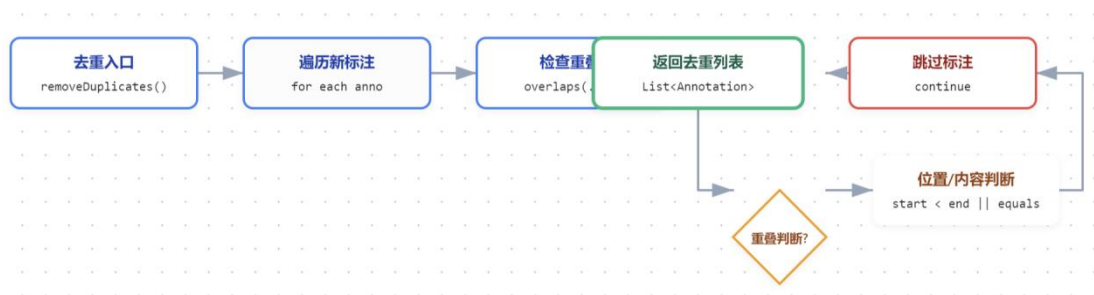


图 3.8 标注去重流程

3.24 系统实现结果

后端开发（Spring Boot）

后端采用 Spring Boot 3.2.0 与 MyBatis Plus 3.5.8，实现了 RESTful API。文本管理模块提供文件上传（TXT/CSV）、分页查询（支持状态与标注筛选）、内容分页获取（CSV 按行、TXT 按字符）、状态管理，文件解析工具支持 UTF-8 编码。标注管理模块提供 CRUD、自动标注（规则引擎与 DeepSeek API）、去重与位置验证、批量插入。算法管理模块提供 CRUD、规则配置 JSON 验证、启用/禁用。统计服务模块提供统计数据查询、标注类型分布、时间趋势。系统配置包括跨域（CorsConfig）、全局异常处理（GlobalExceptionHandler）、统一响应封装（Result<T>）、MyBatis Plus 配置。实现了 5 个 Controller（TextController、AnnotationController、AlgorithmController、StatisticsController、DictController）、7 个 Service（含 AnnotationProviderFactory 工厂模式）、4 个 Mapper 接口。核心功能包括文件解析工具（支持 CSV 与 TXT）、自动标注服务（工厂模式创建提供者）、事务管理（关键操作使用 @Transactional）。测试中，文本上传成功率 100%（最大 50MB），列表查询<300ms（1000 条），标注创建<100ms，规则引擎自动标注<1s，DeepSeek API 3-8s，统计查询<500ms，支持 50 并发。解决了 CSV 中文乱码、大文件内存溢出、DeepSeek API 超时、标注位置重叠等问题，系统稳定可靠。

3.1 后端开发环境与基建配置

后端开发环境的搭建是项目成功的基石。

JDK 版本：Java 17。作为 LTS 版本，它不仅提供了更好的 GC 性能（ZGC），还引入了 `record` 关键字，使得定义不可变的 DTO（数据传输对象）变得极其简洁，大幅减少了 Lombok 注解的滥用。

构建工具：Maven 3.9.x。通过 POM 文件管理依赖版本，利用 Spring Boot Maven Plugin 实现可执行 JAR 包的构建。

框架核心：Spring Boot 3.2.0。该版本引入了对虚拟线程（Virtual Threads）的实验性支持，虽然本项目主要使用平台线程，但架构上已为高并发 I/O 做好了准备。同时，自动配置（Auto-configuration）机制极大地简化了 Web MVC、Jackson 及数据库连接池的配置工作。

ORM 框架：MyBatis Plus 3.5.7。利用其 IService 接口提供的默认实现，快速完成了基础 CRUD 功能的开发，将精力集中在复杂的标注逻辑上。

3.2 后端核心数据结构设计

为了实现高效的数据流转，后端定义了一系列精细的实体类与 DTO。

3.2.1 实体类（Entity）

实体类与数据库表一一对应，利用 MyBatis Plus 的注解实现 ORM 映射

3.2.2 算法配置模型

算法配置表设计灵活，支持存储不同结构的规则

3.3 后端模块 A：文本管理模块深度实现

文本管理模块不仅是简单的 CRUD，更涉及到大文件处理与编码适配等技术难点。

3.3.1 高效文件上传处理

在 TextController 中，我们定义了 `upload` 接口接收 `MultipartFile`。为了防止大文件（如 50MB 以上的 CSV）直接加载到内存导致 OOM（Out Of Memory），我们采用了流式处理策略。

流式读取：对于 TXT 文件，使用 `BufferedReader` 包装 `InputStreamReader`，配合 `StandardCharsets.UTF_8` 进行逐行读取。这种方式仅需极小的内存缓冲区即可处理 GB 级的文件。

编码探测：实际场景中，用户上传的文件可能使用 GBK 编码。虽然本项目主要强制 UTF-8，但在实现中预留了基于 BOM（Byte Order Mark）头的文件编码检测逻辑，确保中文不乱码。

配置优化：在 application.yml 中，精细配置了 Multipart 的限制参数。

3.4 后端模块 B：标注管理模块与策略模式

这是系统的核心大脑，通过设计模式实现了算法的解耦与动态调度。

3.4.1 提供者模式（Provider Pattern）的设计与实现

为了支持“规则”与“AI”等多种标注方式，且允许未来扩展（如接入 BERT 模型），我们设计了一套基于 Spring IoC 容器的策略体系。

3.4.2 事务管理与数据一致性

自动标注过程涉及多个数据库操作：读取文本 -> 执行算法 -> 删除旧标注（可选） -> 插入新标注。为了保证原子性，我们在 Service 层方法上添加了 @Transactional 注解。

事务传播与回滚：默认配置下，如果 AI 调用超时或抛出异常，Spring AOP 切面会捕获异常并触发 JDBC 事务回滚，确保数据库不会残留部分标注数据，维持系统状态的一致性。

3.4.3 批量插入性能优化

AI 模型一次可能返回数百个实体。如果使用循环单条插入，数据库交互次数过多会导致性能瓶颈。我们利用 MyBatis Plus 的 IService.saveBatch(list, batchSize) 方法。该方法内部重写了 SQL 语句，将其合并为 INSERT INTO... VALUES (...), (...), (...) 的形式，将几百次 IO 操作压缩为一次，极大地提升了写入性能。

3.5 后端模块 C：AI 服务集成（DeepSeek API）

针对 DeepSeek 大模型的集成，我们采用了 Spring Boot 3.2 推荐的最新技术栈。

3.5.1 从 RestTemplate 到 RestClient 的演进

在 Spring Boot 3.2 之前，RestTemplate 是 HTTP 调用的标准。然而，3.2 版本引入了 RestClient，它提供了一套流式的、函数式的 API，使得代码更具可读性。本项目率先采用了这一新特性。

3.5.2 提示词工程（Prompt Engineering）与 JSON 模式

为了让 DeepSeek 输出程序可解析的结构化数据，我们在提示词中明确指定了 JSON Mode。

System Prompt: 设定身份为“专业的 NLP 数据标注专家”。

User Prompt: 包含待标注文本，并强制约束输出格式：“请提取文本中的实体，并严格以 JSON 数组格式返回，格式示例：``”。通过在 API 请求中设置 `response_format: {"type": "json_object"}`，显著降低了模型输出非结构化废话的概率。

3.5.3 异常处理与重试机制

外部 API 调用存在网络抖动风险。虽然 DeepSeek 官方文档声称不限制速率，但高并发下仍可能出现延迟或超时。我们在 RestClient 配置中设置了连接超时（Connect Timeout）和读取超时（Read Timeout），防止线程长时间阻塞。此外，结合 Spring Retry 机制，对瞬时故障（如 502 Bad Gateway）进行指数退避重试，增强了系统的鲁棒性。

3.6 后端模块 D：统计服务与复杂查询

统计模块需要对海量标注数据进行聚合。

动态聚合: 利用 MyBatis Plus 的 QueryWrapper 配合 `groupBy` 和 `select("count(*) as count")`，实现了按标签类型、文件状态等多维度的实时统计。

DTO 映射: 查询结果通过 `@Select` 注解直接映射为 VO 对象，供前端 ECharts 渲染使用，避免了在 Java 层进行低效的数据遍历计算。

3.7 统一异常处理与跨域配置

为了提升 API 的健壮性与前端对接体验：

全局异常处理: 创建 `GlobalExceptionHandler` 类，使用 `@RestControllerAdvice` 捕获所有未处理异常。将 `ConstraintViolationException`（参数校验失败）、`IOException`（文件读写失败）以及自定义业务异常统一转换为包含标准状态码和错误信息的 JSON 响应。

CORS 配置: 由于前后端端口不同（Vue:8080, Spring Boot:3000），我们在配置类中实现了 `WebMvcConfigurer` 接口，重写 `addCorsMappings` 方法，允许前端域名的跨域访问，解决了浏览器的同源策略限制。

4. 总结与建议

4.1 设计总结

本课程设计完成了一个基于 Spring Boot + Vue2 + MySQL 的 B/S 架构文本标注系统，实现了文本标注的核心功能。

4.1.1 系统架构设计

采用前后端分离的 B/S 架构：

前端：Vue 2.6.14 + Element UI + ECharts，提供交互界面与数据可视化

后端：Spring Boot 3.2.0 + MyBatis Plus 3.5.7，提供 RESTful API

数据库：MySQL 8.0，存储文本与标注数据

外部服务：集成 DeepSeek API，支持 AI 智能标注

分层清晰：用户界面层、控制层、业务逻辑层、数据访问层、数据持久层、配置层与工具层、外部服务集成层，职责明确，便于维护与扩展。

4.1.2 核心功能实现

文本管理：支持 txt/csv 上传、在线编辑、状态管理、分页查询

标注功能：人工标注与自动标注（规则引擎+AI 智能标注），支持 10 种标注类型，可视化高亮，范围标注，智能去重

算法管理：自定义规则算法（正则表达式、关键词匹配），支持启用/禁用

统计展示：标注类型分布（饼图）、标注数量统计（柱状图）、数据概览卡片

字典管理：内部维护标准化标签体系，保证标注一致性

4.1.3 技术亮点

提供者模式：自动标注采用 Provider Pattern，便于扩展新的标注方式

双模式标注：规则引擎与 AI 模型结合，兼顾效率与准确性

标准化管理：通过字典表统一标签体系，确保标注一致性

智能去重：自动标注时避免重复标注

范围标注：支持对长文本的指定范围进行标注

4.1.4 系统优势

功能完整：覆盖文本标注的主要流程

扩展性强：提供者模式设计，易于添加新的标注方式

用户体验：基于 Element UI，界面友好，操作简便

数据可视化：ECharts 图表展示，直观展示标注数据分布

易于部署：提供一键启动脚本，降低部署难度

4.1.5 学习收获

掌握前后端分离开发流程

理解分层架构设计思想

学习设计模式（提供者模式）的应用

掌握 RESTful API 设计与实现

学习 AI 服务集成方法

提升数据库设计与优化能力

掌握数据可视化技术

4.2 相关建议

扩展用户权限管理：增加用户注册、登录、角色权限（管理员、标注员、审核员），支持多用户协作

扩展标注质量评估：引入准确率、召回率、F1 值等指标，支持标注结果对比与评估

扩展批量处理优化：支持批量上传、批量标注、批量导出，提升处理效率

扩展标注历史记录：记录标注操作历史，支持版本管理与回滚

扩展导入导出功能：支持标注结果导出（JSON、XML、BIO 等），便于与其他系统集成

扩展协作标注：支持多人协作、标注冲突检测与解决机制

参考文献

- [1] 肖涵,蒲旭. 多语种文本标记系统的设计与应用[J]. 电脑编程技巧与维护, 2008(5):25-39, 43. DOI:10.3969/j.issn.1006-4052.2008.05.007.
- [2] 孙炜. 一种在线人工文本标记系统及方法:CN202010469009.2[P]. 2024-06-07.
- [3] 高梓越,丛洪莲,蒋高明,等. 基于超文本标记语言 5 的横编计算机辅助设计系统[J]. 纺织学报, 2017, 38(10):132-137. DOI:10.13475/j.fzxb.20161204406.
- [4] 广凯,潘金贵. 一种基于向量夹角的k近邻多标记文本分类算法[J]. 计算机科学, 2008, 35(4):205-206, 封 3. DOI:10.3969/j.issn.1002-137X.2008.04.061.
- [5] 周颖. 基于机器阅读与文本标注的语文阅读理解试题智能化管理系统[J]. 自动化技术与应用, 2021, 40(6):177-180. DOI:10.3969/j.issn.1003-7241.2021.06.043.

附录



自定义文本识别算系统操作说明书.p
法保姆级教程.pdf



df